DRAFT Fractional covering problems on-line on trees

Bartosz Bromblik

Abstract

1 Introduction

In classic covering problems (vertex cover, independent set, etc.) each vertex of a graph is either included in *the set* or not, which can also be expressed as giving each vertex a value of 1 or 0, respectively. Then, problem requirements are set as some inequalities with those vertices' values. Ex. in vertex cover we require that for each edge, sum of it's endpoints' values sum up to at least 1.

In fractional coverings we use a less strict requirement, namely that each value is a real number in range $[0, 1]$. Ex. in vertex cover, as above, for each edge (u, v), $|u| + |v| \geq 1$. Obviously, every *normal* covering is also a fractional covering.

In this paper we only consider tree covering problems: vertex cover (VC) and independent set (IS). In both scenarios, optimal fractional solutions are the same as *classic* ones (simple proof left as an exercise for the reader, not true for all graphs).

We define *score* of a covering as the *sum of all vertices in the current covering* divided by the *sum of all vertices in on optimal covering*. In VC scores are ≥ 1 and in IS they are ≤ 1. In both problems, for a given solution (covering), the closer the score is to 1 (to optimal solution), the *better* that solution is.

In *on-line* covering problem we imagine two players: algorithm and adversary. The former changes values in vertices to meet problem requirements while also keeping the score as good as possible. The latter builds the graph by adding one edge at a time, trying to make the solution worse.

To make the problem more interesting, we require that all changes made by the algorithm have the same monotonicity, ie.:

- in VC values start at 0 and can only be **increased**, up to 1,
- in IS values start at 1 and can only be **decreased**, down to 0.

2 Linear programming solutions

We focus on VC formulation [with IS versions in square brackets]. $v_i^{(t)}$ stands for value of vertex i after t-th move (adding t edges to the tree).

In this scenario we play as the adversary and we try to maximise [minimise] the score achieved by **any** algorithm.

We start with a list of edges (of length T) we want to add in each turn and try to encode that as an input to some linear programming solver. Keep in mind that even though the algorithm (here: lp solver) knows the whole tree (and all tricks used) in advance, it still has to assert that the covering is correct after each turn.

2.1 Formulation

General List of things we put into our linear programming problem (for a given list of edges):

1. Declare score, which we want (the algorithm) to minimise [maximise].
2. Define limits on all values. For all vertices i, $v_i^{(0)} \geq 0$ and $v_i^{(T)} \leq 1$ [$v_i^{(0)} \leq 1$ and $v_i^{(T)} \geq 0$].
3. Force monotonicity of all values. For each vertex i and each turn t (from 1 to T) $v_i^{(t-1)} \leq v_i^{(t)}$ [$v_i^{(t-1)} \geq v_i^{(t)}$].
4. Add edges. When we want to connect vertices i and j in turn t, we write $v_i^{(t)} + v_j^{(t)} \geq 1$ [$v_i^{(t)} + v_j^{(t)} \leq 1$].
5. Compare with optimal. After each turn t we add $\sum_i v_i^{(t)} \leq Opt^{(t)} * score$ [$\sum_i v_i^{(t)} \geq Opt^{(t)} * score$], where $Opt^{(t)}$ stands for the sum of vertex values in the optimal solution.

Symmetry Breaking In a *casual game* it is alway beneficial for the adversary to add the second edge in each connected component to the vertex with lower [higher] value, because it forces the algorithm to increase the total sum by more (for edge with values $a \leq b$, connecting the third vertex to the first one results in the grand total of $1 + b$, which is greater or equal to $1 + a$, which we get by doing the other move).

Therefore, when we see a second edge (v_a, v_c) is begin added to a connected component $\{v_a, v_b\}$ (at time t), we write that $v_a^{(t)} \leq v_b^{(t)}$ [$v_a^{(t)} \leq v_b^{(t)}$]. We call that *Symmetry Breaking* (or *SB*, *BS*).

Branching We allow ourselves to consider and encode multiple ways of building the tree. More precisely, at turn t we add some edge in branch p and another one in branch q. Keep in mind that branches cannot be merged.

One can think of them as an option for the adversary to change tactic depending on the state of the tree. Or just an another possibility the algorithm needs to account for when assigning values.

To denote multiple branches we can use $v_i^{(t,b)}$ where b is the number of branch that vertex belongs to (in turn t). Multiple branches can come from the same state, but they are completely independent, so we can encode on them one by one, using just their indexes b.

As in step 3., we need to assert monotonicity of values. If branch q forks from branch p at turn t, we write $v_i^{(t-1,p)} \leq v_i^{(t,q)}$ $[\geq]$ for each vertex i.

Also, we need to terminate every branch, so we again write $v_i^{(T_b,b)} \leq 1$ $[... \geq 0]$ for each vertex, where T_b is the last turn in branch b.

It's worth noting, that a complete solution would consider all possible branches. But our goal here is to achieve the best possible score with a reasonable amount of computation.

Attacks An attack on vertex v is a way to force a value of 1 in this vertex [not practical in IS]. We do that by connecting *a lot* of edges to that vertex.

We always attack multiple vertices at once. More precisely, after each turn, for every correct *normal* vertex cover S of the tree, we attack all of the vertices in S.

We encode that similarly to step 5.4. Namely, we write $|S| + \sum_{v_i \notin S} v_i^{(t)} \leq |S| * score$.

2.2 Results

After excessive testing we found that a good strategy, both in terms of score and computation time, is a line (vertex degree at most 2).

We start with one edge. We say it has a left and right end. Next, we add an edge on the right, breaking the symmetry. Later, we add one edge on the right, one on the left. One on the left and one on the right. And so on.

We do not use branching. The only attack used is for the optimal covering after an even number of turns (odd number of vertices).

With such means we achieved a score of over 1.69.

3 Vertex Cover lower bound

General Only important inequalities are those representing optimal vertex covers when the path is of odd length.
First important inequalities are:

$$x + z = \alpha$$

$$Y + 2 \cdot a_0 = 2 \cdot \alpha$$

$$X + Z + 2 \cdot a_1 = 3 \cdot \alpha \quad \equiv \quad 2 \cdot (1 - a_0) + 2 \cdot a_1 = 3 \cdot \alpha$$

$$Y + 2 \cdot (1 - a_1) + 2 \cdot a_2 = 4 \cdot \alpha$$

$$2 \cdot (1 - a_0) + 2 \cdot (1 - a_2) + 2 \cdot a_3 = 5 \cdot \alpha$$

Assumptions

- Every important inequality is treated as equality to make the example easy to solve.
- Symmetry breaking: second edge is connected to the vertex with smaller value (or arbitrarly if $x = y$).

Calculations First let's assume the optimal start of $x = y = \frac{1}{2}$.
Next we need $Y + z = 1$ and $x + z = \alpha$. Therefore $z = \alpha - \frac{1}{2}$ and $y = \frac{3}{2} - \alpha$.
Next we have $2 \cdot a_0 + Y = 2 \cdot \alpha$, from which we get $a_0 = \frac{3}{2}\alpha - \frac{3}{4}$, and $X = Z = 1 - a_0 = \frac{7}{4} - \frac{3}{2}\alpha$.
Next $2 \cdot (1 - a_0) + 2 \cdot a_1 = 3 \cdot \alpha$, therefore $a_1 = 3 \cdot \alpha - \frac{7}{4}$.

Further, we notice a pattern that every ith and $i + 2$th equations look like:

$$(sth) + 2 \cdot a_k = (k + 2) \cdot \alpha$$

$$(sth) + 2 \cdot (1 - a_{k+1}) + 2 \cdot a_{k+2} = (k + 4) \cdot \alpha$$

We subtract the former from the latter and we get:

$$a_{k+2} + 1 - a_{k+1} - a_k = \alpha \qquad \Rightarrow \qquad a_{k+2} = a_{k+1} + a_k + (\alpha - 1)$$

Calculating first few terms to notice the pattern:

$$a_0 = 1 \cdot (a_0) + 0 \cdot (a_1) + 0 \cdot (\alpha - 1)$$

$$a_1 = 0 \cdot (a_0) + 1 \cdot (a_1) + 0 \cdot (\alpha - 1)$$
$$a_2 = 1 \cdot (a_0) + 1 \cdot (a_1) + 1 \cdot (\alpha - 1)$$
$$a_3 = 1 \cdot (a_0) + 2 \cdot (a_1) + 2 \cdot (\alpha - 1)$$
$$a_4 = 2 \cdot (a_0) + 3 \cdot (a_1) + 4 \cdot (\alpha - 1)$$
$$a_5 = 3 \cdot (a_0) + 5 \cdot (a_1) + 7 \cdot (\alpha - 1)$$

We can spot the Fibonacci series here, which we define as $F_0 = 0$, $F_1 = 1$, $F_{i+2} = F_{i+1} + F_i$.
For $i \geq 1$:

$$a_i = F_{i-1} \cdot a_0 + F_i \cdot a_1 + (F_{i+1} - 1) \cdot (\alpha - 1)$$

Which is obvious if you thing about it.

We can now substitute a_0 and a_1 with their respective values:

$$a_i = F_{i-1} \cdot \left(\frac{3}{2}\alpha - \frac{3}{4}\right) + F_i \cdot \left(3 \cdot \alpha - \frac{7}{4}\right) + (F_{i+1} - 1) \cdot (\alpha - 1)$$

$$= \frac{1}{4}\left(\alpha \cdot (6 \cdot F_{i-1} + 12 \cdot F_i + 4 \cdot F_{i+1} - 4) - (3 \cdot F_{i-1} + 7 \cdot F_i + 4 \cdot F_{i+1} - 4)\right)$$

$$= \frac{1}{4}\left(\alpha \cdot (6 \cdot F_i + 10 \cdot F_{i+1} - 4) - (4 \cdot F_i + 7 \cdot F_{i+1} - 4)\right)$$

We assumed that $\forall_i \; a_i \geq 0$, so:

$$a_i = \frac{1}{4}\left(\alpha \cdot (6 \cdot F_i + 10 \cdot F_{i+1} - 4) - (4 \cdot F_i + 7 \cdot F_{i+1} - 4)\right) \geq 0$$

Then:

$$\alpha \geq \frac{4 \cdot F_i + 7 \cdot F_{i+1} - 4}{6 \cdot F_i + 10 \cdot F_{i+1} - 4}$$

We now calculate the limit, knowing that $\lim_{i\to\infty} \frac{F_{i+1}}{F_i} = \phi = \frac{1+\sqrt{5}}{2}$:

$$\lim_{i\to\infty} \frac{4\cdot F_i + 7\cdot F_{i+1} - 4}{6\cdot F_i + 10\cdot F_{i+1} - 4} = \lim_{i\to\infty} \frac{4 + 7\cdot \frac{F_{i+1}}{F_i} - \frac{4}{F_i}}{6 + 10\cdot \frac{F_{i+1}}{F_i} - \frac{4}{F_i}} = \frac{4 + 7\cdot\phi}{6 + 10\cdot\phi} = \frac{5-\sqrt{5}}{4} = \frac{3-\phi}{2}$$

We again drop the inequality to get $\alpha = \frac{3-\phi}{2}$. From this we calculate:

$$a_0 = \frac{3}{2}\alpha - \frac{3}{4} = \frac{6 - 3\cdot\phi}{4}$$

$$a_1 = 3\cdot\alpha - \frac{7}{4} = \frac{11 - 6\cdot\phi}{4}$$

We now substitute those to our general formula for a_i and get:

$$a_i = F_{i-1}\cdot\frac{6-3\cdot\phi}{4} + F_i\cdot\frac{11-6\cdot\phi}{4} + (F_{i+1}-1)\cdot(\frac{3-\phi}{2} - 1)$$

...TODO...

$$a_i = \frac{1}{4}\left(F_{i+6} - \phi\cdot F_{i+5} + 2\cdot\phi - 2\right)$$

4 Independent Set bounds